

OSPFI: One-Shot Skill Learning for Part-Level Fine-grained Interaction in Robotic Manipulation

Abstract—Learning from a single demonstration is an efficient way for robots to acquire and transfer new manipulation skills. Existing methods mainly focus on object-level interactions, but can not sufficiently address part-level interactions in fine-grained manipulation tasks. In this work, we propose OSPFI, a one-shot skill learning framework for part-level interaction in robotic manipulation. From a single demonstration, OSPFI effectively represents the part-interaction skill as an interaction template that records the spatial relationship between the parts at interaction start frame, the task-critical interaction location, and the subsequent interaction process. During inference, OSPFI can generalize the learned manipulation skill across geometrically diverse objects with corresponding parts through coarse-to-fine tool part alignment. We evaluate OSPFI on three real-world manipulation tasks involving different tool and target object instances. Experimental results show that OSPFI achieves higher task success rates than baselines and demonstrates strong generalization to unseen object combinations. Project website: <https://sites.google.com/view/ospfi-project>.

I. INTRODUCTION

Learning from few demonstrations enables robots to efficiently acquire new skills and adapt to new environments [1], [2]. This is especially valuable for tool manipulation, where collecting large amounts of task-specific demonstrations can be laborious. The ability to learn tool manipulation skills from a single demonstration therefore provides a simple and efficient way to extend the range of tasks that robots can perform [3], [4], [5]. To use a tool effectively, the robot must understand not only the overall object configuration but also how the relevant regions of the tool and the target object should be arranged during interaction.

Previous few-shot robot manipulation methods have explored transferring manipulation skills from limited demonstrations to new scenes. These methods commonly enable robots to accomplish equivalent tasks through geometric alignment [6], [7], feature-based correspondence [8], [9], and global trajectory transformation [10], [11]. Such approaches can perform well on relatively simple tool manipulation tasks (e.g., *pouring water from a mug into a bowl*). However, for some fine-grained interaction tasks, only a small portion of each object may directly contribute to task completion. For example, hanging a mug requires the mug handle to be properly positioned with respect to the peg on the hanging rack. In such tasks, the spatial relationship between the interacting parts can be more important than the global geometric relationship between the entire objects [12]. When the target object changes significantly in shape or size, the relative location of the target part on the object may also change. In such cases, object-level pose alignment and trajec-

tory transformation may not accurately preserve the spatial relationship required for part interaction. Moreover, when the geometry of the tool differs considerably from that in the demonstration, the alignment based mainly on geometric similarity may fail to align the functionally critical region of the tool with the correct target location for completing the task, resulting in task execution failure.

Recent studies have started to use part-level and functional representations to improve the performance of cross-instance skill transfer in robot manipulation. Some methods explicitly model part-level relational structures, such as inter-object part correspondences or local coordinate frames on task-relevant parts, to represent manipulation relationships [13], [14]. These approaches can improve generalization across object instances, but usually require task-specific training with demonstrations. Other methods exploit functional correspondences to transfer manipulation skills across objects with different geometries or appearances [15], [16]. However, they mainly focus on establishing functional correspondences across objects for skill transfer, rather than explicitly modeling the spatial interaction relationship between the interacting parts. Directly extracting this relationship from a single demonstration and transferring it to novel tool and target instances without additional task-specific training has not been sufficiently studied.

Inspired by these observations, we propose OSPFI, a one-shot skill learning framework for part-level fine-grained interaction tasks. In the offline stage, OSPFI first identifies the interaction-start keyframe and then extracts the tool part, target part, their semantic features, and the functional interaction keypoint at this keyframe. These elements, together with the subsequent interaction trajectory, are organized into a part-interaction template that represents the spatial relationship between the interacting parts and the manipulation process. During inference, OSPFI first aligns the target part in the current scene to determine the desired initial interaction position of the tool part. It then performs coarse-to-fine tool part alignment to ensure that the critical interaction region and orientation of the tool are correctly aligned with the target part. We evaluate the proposed method on three challenging real-world tasks: sweeping, hanging and pouring. Compared with existing skill transfer and manipulation methods, OSPFI achieves higher task success rates across different object instances and scene configurations, demonstrating strong generalization in part-level functional interaction tasks.

In summary, the main contributions of this work are as follows:

- We propose a part-interaction skill representation that organizes the spatial relationship between the interacting parts, the functional interaction keypoint, and the interaction trajectory into a compact interaction template for one-shot skill transfer.
- We propose a part-level interaction transfer method consisting of target part alignment and coarse-to-fine tool part alignment to accurately transfer the demonstrated interaction relationship to new object instances.
- We conduct extensive real-robot experiments, demonstrating that OSPFI exhibits strong generalization capability compared with existing baselines.

II. RELATED WORK

One-Shot Skill Learning. One-shot skill learning enables robots to generalize manipulation skills from a single demonstration. Depending on different application scenarios, existing one-shot methods have adopted different strategies to improve generalization and manipulation performance [17], [18]. IMOP [19] focuses on general one-shot manipulation in novel scenes by identifying and matching task-invariant regions to determine robot actions. MAGIC [20] focuses on contact-rich manipulation across objects with different geometries, transferring strategies through global shape matching and local contact analogy. ODIL [21] addresses coordinated dual-arm manipulation by parameterizing trajectories from a single demonstration and using visual servoing for robust execution. In contrast, OSPFI focuses on tool manipulation between specific regions of tool and target object, and explicitly transfers their spatial interaction relationship to novel instances.

Fine-grained Operation Skill Learning. Recent studies have increasingly adopted part-level or functional-region representations to improve performance in fine-grained tasks. Some studies use affordance segmentation [22], [23], [24] to identify task-relevant object regions for manipulation planning. SAGE [25] links semantic parts with actionable regions to identify functional regions for manipulation. PartInstruct [26] further studies fine-grained manipulation with part-level instructions and reasons about object parts and their task relationships. Compared with these methods, OSPFI not only identifies task-relevant part regions, but also explicitly models the spatial interaction relationship between the functional regions of the tool and target object.

Keypoint Representations for Manipulation. Keypoint representations provide a compact way to support manipulation planning and generalization [27], [28], [29]. kPAM [30] uses semantic 3D keypoints and geometric constraints for category-level manipulation, while KETO [31] learns task-relevant tool keypoints for grasping and manipulation. ReKep [32] represents semantic keypoints as relational constraints for multi-stage motion generation. However, these methods mainly focus on object properties or manipulation constraints rather than the interaction relationship between the manipulating tool and the manipulated object. OSPFI instead combines tool and target part points with a functional interaction keypoint to represent the interaction configuration.

These elements are jointly used for alignment across object instances.

III. PROBLEM FORMULATION

This work studies one-shot part-interaction robot skill learning. Given a single demonstration of a specific part-interaction task involving a tool part and a target part, the goal is to enable the robot to perform an equivalent task in a similar scene using tools of the same category. Specifically, the robot needs to grasp the tool with a parallel-jaw gripper and then use it to interact with a specific part of another target object.

The demonstration is a record of the robot’s manipulation process, which is represented as $D_H = \{(O_t, a_t)\}_{t=0}^{N-1}$. Here, D_H denotes the whole demonstration, N is the total number of frames, O_t is the RGB-D observation captured by a stationary camera at timestep t , and $a_t = (R_t, \mathbf{t}_t, g_t)$ represents the corresponding robot action, where $R_t \in SO(3)$ and $\mathbf{t}_t \in \mathbb{R}^3$ denote the 3D orientation and translation of the end effector respectively, and g_t denotes the gripper state (open or closed). In addition, each demonstration is paired with a task description, such as “*use the brush to clean the spout of the teapot*”.

IV. OSPFI

In this section, we introduce OSPFI, a one-shot strategy for part-level fine-grained manipulation skill learning. As shown in Fig. 1, OSPFI mainly consists of two stages. (1) Part-Interaction Template Extraction: OSPFI extracts an interaction template from a single demonstration to represent the spatial relationship between the interacting parts, the task-critical interaction location, and the manipulation process. (2) Part Alignment: During inference, OSPFI transfers the learned interaction relationship to the current scene by transferring the target part and then performing coarse-to-fine tool part alignment, enabling the demonstrated skill to generalize across objects with different geometries and spatial configurations.

A. Part-Interaction Template Extraction

Interaction-Start Frame Extraction. OSPFI extracts the part-interaction template from the trajectory after the gripper closes. This trajectory consists of an approach phase and an interaction phase. In the former, the grasped tool moves substantially toward the target part, while in the latter, the tool part interacts with the target part. We define the boundary between these two phases as the interaction-start frame t_{split} . To identify this frame, we compute the motion energy between adjacent RGB frames following the method in [33] and select the frame with the steepest decrease in motion energy as t_{split} . The corresponding RGB-D observation is denoted as $O_{t_{split}}$.

Part Segmentation Based on VLM and Grounded SAM.

We assume that the target part remains unchanged after the robot grasps the tool. To avoid occlusion, the tool part and target part are segmented from the RGB image at t_{split} and the RGB image captured after gripper closure, respectively.

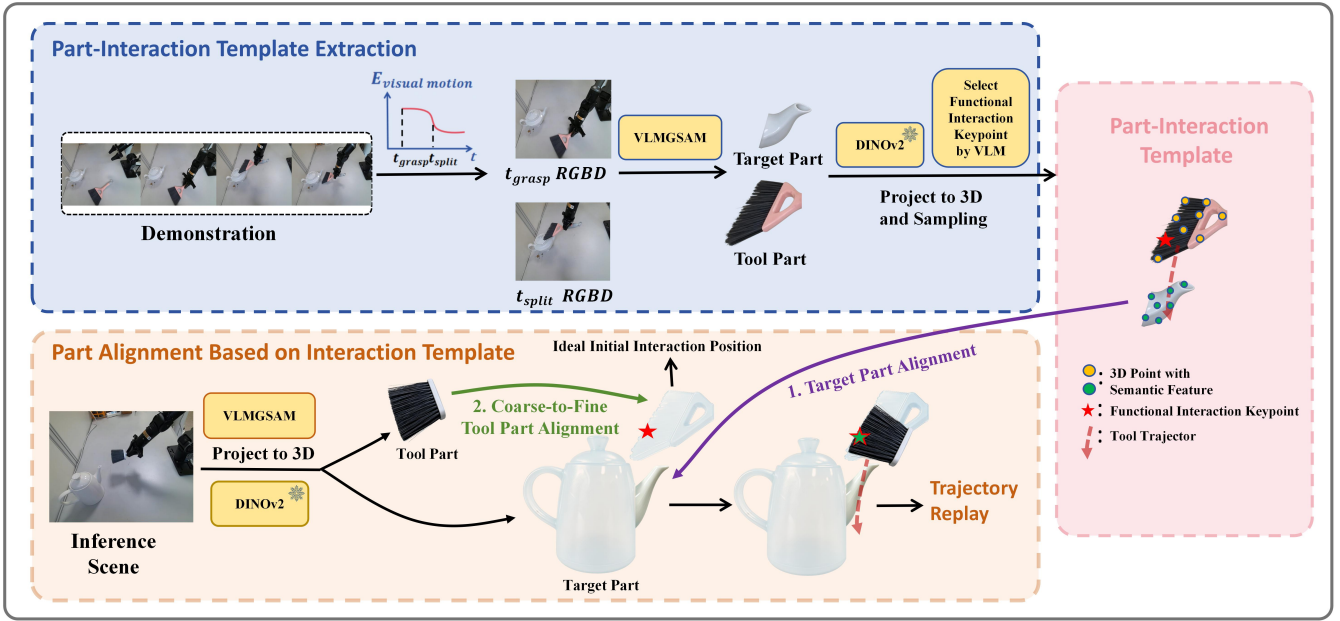


Fig. 1. Overview of the OSPFI framework. OSPFI consists of two stages: Part-Interaction Template Extraction and Part Alignment. The first stage corresponds to offline interaction skill learning, while the second stage corresponds to online skill transfer during inference.

Existing open-vocabulary part segmentation models, such as VLPART [34] and PartCLIPSeg [35], generally perform well on typical object parts, such as mug handles and brush handles. However, interaction-related target parts may occupy only a small region of the object. Their names or appearances may vary across instances or be ambiguous, making direct segmentation prone to failure. Considering the understanding capability of vision-language model (VLM), OSPFI proposes a two-stage part segmentation pipeline combining the VLM with Grounded SAM [36], [37] to improve the robustness of part segmentation (abbreviated as VLMGSAM).

Fig. 2 illustrates the overall VLMGSAM pipeline. First, we use Grounded SAM to obtain the bounding box and mask of the entire object containing the part. We then apply Farthest Point Sampling (FPS) [38] within the object mask to uniformly sample N_c candidate points, each of which is labeled with an index. Next, both the original image and the image containing the object mask and indexed candidate points are provided to the VLM. The VLM identifies the indices of the points belonging to the target part. We treat the selected points as positive points and the remaining candidate points as negative points. The initial bounding box generated by GroundingDINO is then tightened according to the spatial distribution of the positive points while ensuring that all positive points remain inside the box. Finally, we use the positive and negative points together with the refined bounding box as spatial prompts and provide the detailed part name as the text prompt to SAM to obtain the final part mask.

Part-Interaction Template with Semantic Point Clouds and Functional Interaction Keypoints. OSPFI represents the spatial relationship between interacting parts using semantic point clouds of the tool and target parts at t_{split} . Based on the part masks obtained by VLMGSAM, we project

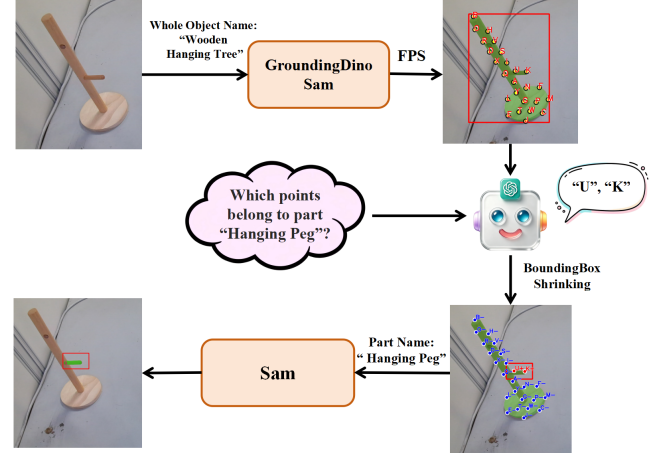


Fig. 2. An illustration of the VLMGSAM pipeline.

masked pixels into 3D space using the depth image and camera extrinsics to construct the corresponding part point clouds, and then apply FPS to reduce point numbers. Next, we use the pretrained DINOv2 [39] model to extract dense semantic features for the tool and target part points by bilinearly interpolating the feature map at the corresponding pixel location of each 3D point.

However, the above point clouds only reflect the geometry of the parts used in the demonstration. When transferring to tools with different geometries during inference, it is important to identify and align their task-critical regions. To improve generalization across tools with different shapes, OSPFI selects a functional interaction keypoint from the tool part point cloud. This point is typically located at the position on the tool part that is closest to or interacts most closely with the target part, which is critical to the completion of interaction. As shown in Fig. 3, we uniformly sample several RGB images from the interaction process starting at t_{split} . These images and the RGB image annotated with

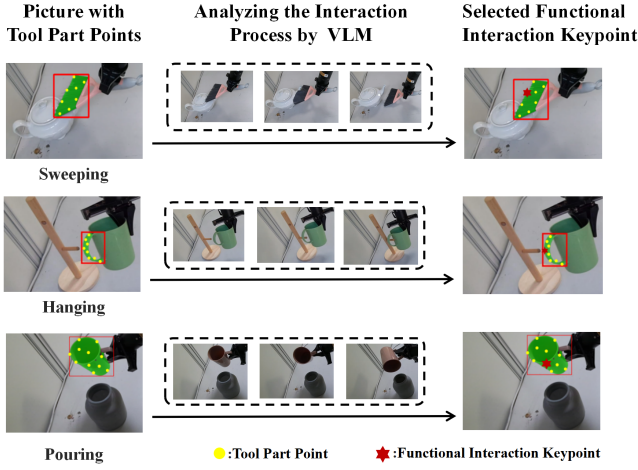


Fig. 3. An illustration of VLM-based functional interaction keypoint selection by analyzing interaction process.

the 2D tool part points are provided to the VLM to select the functional interaction keypoint. During inference, the functional interaction keypoint is used to achieve functional alignment across tools with different geometries, which is critical for improving cross-instance generalization. We will describe this process in Section IV-B.

The part-interaction template contains the following items:

- The tool name and tool part text name.
- The target object name and target part text name.
- The tool part point cloud $\mathbf{P}_{tool} \in \mathbb{R}^{N_{tool} \times 3}$, its corresponding feature $\mathbf{F}_{tool} \in \mathbb{R}^{N_{tool} \times C}$, and the functional interaction keypoint index k_{func} .
- The target part point cloud $\mathbf{P}_{target} \in \mathbb{R}^{N_{target} \times 3}$ and its corresponding feature $\mathbf{F}_{target} \in \mathbb{R}^{N_{target} \times C}$.
- The interaction trajectory, represented by relative end-effector transformations $\{\Delta \mathbf{T}_t^{ee}\}_{t=t_{split}+1}^{N-1}$ after t_{split} , where $\Delta \mathbf{T}_t^{ee} = \mathbf{T}_t^{ee} (\mathbf{T}_{t-1}^{ee})^{-1} \in SE(3)$ denotes the relative pose transformation of the end-effector from time $t-1$ to t .

B. Part Alignment Based on Interaction Template

In this section, we present the core component of OSPFI, which transfers and generalizes the demonstrated manipulation skill to new scenes through target part and coarse-to-fine tool part alignment based on interaction template.

Target Part Alignment. After the tool is stably grasped and the RGB-D image is obtained, OSPFI determines the ideal interaction start pose of the tool part by locating the target part in the current scene. First, based on the target object text name and target part text name, we adapt VLMGSAM to obtain the target part mask from the RGB image, then extract its point cloud and semantic features using the same procedure as in template extraction, denoted as $\mathbf{P}_{target}^{curr}$ and $\mathbf{F}_{target}^{curr}$ respectively. We then calculate the transformation from the template target part to the current target part, which is achieved through point cloud alignment based on semantic similarity by solving the following optimization problem:

$$\begin{aligned} & (s_{target}, R_{target}, t_{target}, \{i_k\}_{k=1}^{N_{target}}) \\ & = \arg \min_{s, R, t, \{i_k\}} \sum_{k=1}^{N_{target}} \left[\|p_{i_k} - (sR\hat{p}_k^{target} + t)\|_2^2 + \lambda \text{dis}(f_{i_k}, \hat{f}_k^{target}) \right] \end{aligned} \quad (1)$$

where

$$\text{dis}(f_{i_k}, \hat{f}_k^{target}) = \begin{cases} d(f_{i_k}, \hat{f}_k^{target}), & d(f_{i_k}, \hat{f}_k^{target}) \leq \delta_f, \\ +\infty, & d(f_{i_k}, \hat{f}_k^{target}) > \delta_f. \end{cases} \quad (2)$$

In the above equations, s_{target} , R_{target} , and t_{target} denote the calculated scale, rotation, and translation from the template target part to the current target part. $\hat{p}_k^{target}$ and $\hat{f}_k^{target}$ denote the 3D coordinate and semantic feature of the k -th template target part point, while i_k denotes the matched point index in the current target part point cloud corresponding to the k -th template target part point, with p_{i_k} and f_{i_k} representing the 3D coordinate and semantic feature, respectively. $d(\cdot, \cdot)$ denotes the L2 feature distance, δ_f is the semantic feature distance threshold, and λ is the weight balancing the geometric matching error and semantic distance. The optimization problem can be solved using the Umeyama algorithm [40]. To handle the problem where some keypoints are occluded and cannot be matched in real scenes, we incorporate RANSAC strategy [41] into the solution process to improve robustness.

Since the Umeyama solution satisfies $c_{target} = s_{target}R_{target}\hat{c}_{target} + t_{target}$, we apply the calculated transformation to the tool part point cloud in the template according to

$$p_{k,ideal}^{tool} = c_{target} + s_{target}R_{target}(\hat{p}_k^{tool} - \hat{c}_{target}) \quad (3)$$

where $\hat{c}_{target} = \frac{1}{N_{target}} \sum_{k=1}^{N_{target}} \hat{p}_k^{target}$ is the center of template target part, $c_{target} = \frac{1}{N_{target}} \sum_{k=1}^{N_{target}} p_{i_k}$ is the center of current target part, and $k = 1, \dots, N_{target}$. The set $\{p_{k,ideal}^{tool}\}_{k=1}^{N_{target}}$ represents the ideal initial interaction position of the tool part in the current scene, which serves as the target for the subsequent tool-part alignment.

Coarse-to-Fine Tool Part Alignment. After obtaining the ideal initial interaction position of the tool part, the goal is to determine the end-effector transformation that moves the grasped tool part from its current position to this ideal position. Since the current tool may be different from the demonstrated tool in shape, point cloud alignment alone may fail to accurately align the task-critical interaction region and orientation, leading to task failure. To improve generalization across tools with different shapes, we propose a coarse-to-fine tool part alignment strategy containing functional alignment, as illustrated in Fig. 4. The method is detailed below.

a) Coarse Alignment Based on Point Cloud Transformation: We first use the VLMGSAM pipeline to obtain the mask of the current tool part from the RGB image and extract its point cloud with semantic features, denoted as $\mathbf{P}_{tool}^{curr}$ and $\mathbf{F}_{tool}^{curr}$ respectively. We then apply the same point cloud alignment based on semantic similarity as in Eqs. (1) and (2)

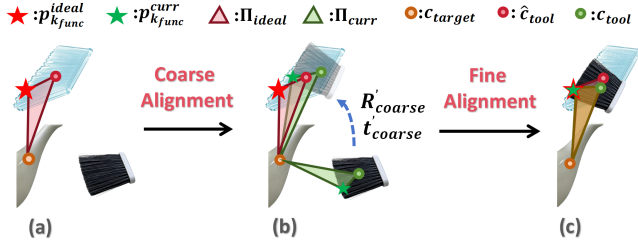


Fig. 4. An illustration of coarse-to-fine tool-part alignment. (a) Ideal initial interaction position of the tool part and current tool part. (b) Coarse matching identifies the corresponding functional interaction point on the current tool and calculates the transformation R'_{coarse} and t'_{coarse} for roughly aligning the tools' shape. (c) Tool and target parts after fine alignment.

to align the ideal tool part point cloud $\{p_{k,ideal}^{tool}\}_{k=1}^{N_{tool}}$ with the current tool part point cloud. This provides the correspondences between the template tool part points and the current point cloud, denoted as $\{j_k\}_{k=1}^{N_{tool}}$, together with the coarse alignment parameters $(s_{coarse}, R_{coarse}, t_{coarse})$. Since the geometry of the current tool remains unchanged, we use only the rotation and translation components and invert the coarse alignment transformation to obtain $(R'_{coarse}, t'_{coarse})$ from the current tool part to the ideal initial interaction position, where $R'_{coarse} = R_{coarse}^T$ and $t'_{coarse} = -R_{coarse}^T t_{coarse}$.

b) Fine Alignment Based on Functional Interaction Key-point: In the fine alignment stage, we further align the tool part based on the functional interaction keypoint. Using the template index k_{func} , we identify the corresponding ideal keypoint $p_{k_{func}}^{ideal}$ in the ideal tool part point cloud and its matched point $p_{k_{func}}^{curr}$ in the current tool part from the coarse-alignment correspondences $\{j_k\}_{k=1}^{N_{tool}}$. The centers of the target part, ideal tool part, and current tool part can be calculated as $c_{target} = \frac{1}{N_{target}} \sum_{k=1}^{N_{target}} p_{i_k}$, $\hat{c}_{tool} = \frac{1}{N_{tool}} \sum_{k=1}^{N_{tool}} p_{k,ideal}^{tool}$, and $c_{tool} = \frac{1}{N_{tool}} \sum_{k=1}^{N_{tool}} p_{j_k}$. We define the interaction plane using the tool part center, target part center, and functional interaction keypoint. Specifically, the ideal interaction plane Π_{ideal} is defined by c_{target} , $\hat{c}_{tool}$, and $p_{k_{func}}^{ideal}$, while the current interaction plane Π_{curr} is defined by c_{target} , c_{tool} , and $p_{k_{func}}^{curr}$.

The objective of the fine alignment stage mainly includes the following two aspects:

- Aligning the current functional interaction keypoint $p_{k_{func}}^{curr}$ with the ideal keypoint $p_{k_{func}}^{ideal}$. This aligns the tool's critical functional region with the desired interaction position.
- Making Π_{curr} parallel to Π_{ideal} , equivalently aligning their normal vectors. This keeps the orientation of the tool part relative to the target part consistent with the demonstration template.

Thus, the optimization problem of the fine alignment stage can be formulated as :

$$\begin{aligned} \min_{R,t} \quad & w_R \left\| \log \left(R'_{coarse}{}^T R \right) \right\|_2^2 + w_t \|t - t'_{coarse}\|_2^2 \\ \text{s.t.} \quad & R p_{k_{func}}^{curr} + t = p_{k_{func}}^{ideal}, \\ & \left| (R n_{curr})^T n_{ideal} \right| = 1. \end{aligned} \quad (4)$$

where n_{curr} and n_{ideal} denote the unit normal vectors of Π_{curr} and Π_{ideal} , respectively. The above optimization satisfies the functional alignment constraints while keeping the final result close to the coarse alignment result. The solution to this optimization problem, R_{fine}^* and t_{fine}^* , represent the final transformation applied to the tool part to move it to the initial interaction pose. Assuming that no relative motion occurs between the gripper and the tool after grasping, the robot's interaction start pose T_{ee}^{start} can finally be calculated by applying this transformation to the current end-effector pose.

C. Execution

During online inference, OSPFI first detects the tool part and uses AnyGrasp [42] to sample a grasp pose outside the interaction region. After grasping, the robot calculates the interaction start pose T_{ee}^{start} following Section IV-B and executes it through inverse kinematics. The end-effector transformations stored in the template are then transferred to the current scene and sequentially executed to complete the task. Overall inference procedure is summarized in Algorithm 1.

Algorithm 1: OSPFI Inference Pipeline

Require: Part-interaction template $\mathcal{T} =$

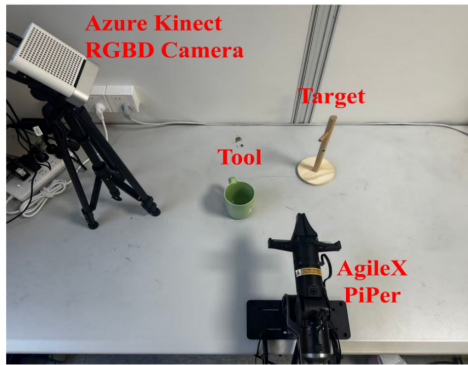
$\{\mathbf{P}_{tool}, \mathbf{F}_{tool}, \mathbf{P}_{target}, \mathbf{F}_{target}, k_{func}, \{\Delta \mathbf{T}_t^{ee}\}\};$
 initial RGB-D observation O_{init} ; VLMGSAM for part segmentation; DINOv2 for semantic feature extraction; AnyGrasp for grasp pose generation

Output: Task execution

1. Use VLMGSAM to obtain tool part mask $M_{tool-part-init}$;
 2. Obtain the grasp pose T_{grasp} outside $M_{tool-part-init}$ using AnyGrasp, and execute it;
 3. Obtain the post-grasp observation $O_{after-grasp}$ and the end-effector pose T_{ee}^{curr} ;
 4. Obtain $\mathbf{P}_{target}^{curr}$, $\mathbf{F}_{target}^{curr}$, $\mathbf{P}_{tool}^{curr}$, and $\mathbf{F}_{tool}^{curr}$ from $O_{after-grasp}$ using VLMGSAM and DINOv2;
 5. Transfer the target part to obtain the ideal tool part point cloud $\mathbf{P}_{tool}^{ideal}$ using Eqs. (1)–(3);
 6. Perform coarse-to-fine tool part alignment to obtain R_{fine}^* and t_{fine}^* using Eqs. (4);
 7. Obtain the interaction start pose T_{ee}^{start} by transforming T_{ee}^{curr} according to R_{fine}^* and t_{fine}^* , then execute it;
 8. Transfer and execute $\{\Delta \mathbf{T}_t^{ee}\}$;
-

V. EXPERIMENTS

In this section, we conduct real-world robot experiments to investigate the skill transfer and generalization performance of OSPFI across different object instances and object positions. The experimental setup is shown in Fig. 5(a). We use a 6-DoF AgileX PiPer robotic arm equipped with a 1-DoF parallel gripper and a fixed Azure Kinect RGB-D world camera to capture visual observations. MoveIt is used for inverse kinematics and robot motion planning. In the execution pipeline of OSPFI, we use GPT-4o as VLM and DINOv2 with the ViT-L/14 backbone. The VLM prompts used in the pipeline are provided on the project website.



(a) Experiment Setup



(b) Object instances and text prompts

Fig. 5. (a) Real-world experiment setup. (b) The tool and target objects in three task scenarios, along with the corresponding text prompts.

We conduct experiments on the following three fine-grained tasks involving interaction between specific parts of the tool and target object. The task description for each scenario is given below:

- **Sweeping:** The task description is “using a brush to clean the spout of a kettle.” A trial is considered successful if the brush bristles make contact with the kettle spout and perform sufficient sweeping motion over it.
- **Hanging:** The task description is “hanging a mug by its handle on a peg of a hanging tree.” A trial is considered successful if the mug handle is stably hung on the peg without causing large movement of the peg during the hanging process.
- **Pouring:** The task description is “using a mug to pour its contents into the opening of a bottle.” A trial is considered successful if the contents of the mug are successfully poured into the opening of the bottle.

To evaluate generalization across object instances, for each task, we test combinations of three tool instances and two target instances with varying shapes. The object instances used in the experiments and the corresponding text prompts are shown in Fig. 5(b). For the two target instances, the target parts are located at different positions within the objects. The text prompts are generated by the VLM from the corresponding images.

A. Comparison with One-Shot Strategies

We compare OSPFI with other one-shot skill learning strategies. The baselines include the following methods:

- **DINO-based Pose-Estimation Transfer [11]:** A trajectory transfer method that uses DINO feature matching

to establish correspondences between objects, calculates their rigid transformation, and transfers the demonstrated end-effector trajectory accordingly.

- **DITTO [10]:** A strategy that transfers the demonstrated manipulation trajectory to new object configurations by warping the original trajectory according to the pose changes of the manipulated object and secondary object from the demonstration scene to the current scene.
- **TReF-6 [43]:** A one-shot skill generalization strategy that infers a 6-DoF task-relevant frame from a single demonstration and transfers the demonstrated trajectory using Dynamic Movement Primitive defined relative to this frame.

For all baselines, we use a single demonstration for each task and evaluate the methods on different combinations of tool and target object instances. We conduct 10 trials for each baseline in every scene combination and record the number of successful trials. In each trial, the tool and target objects are randomly placed within the robot’s workspace.

Table I presents the experimental results of different methods across various object combinations. OSPFI achieves a higher overall success rate than baselines in all three task scenarios. Both DINO-based Pose-Estimation Transfer and DITTO rely on object-level transformations from the demonstration scene to the current scene. These transformations are then used to transfer the entire demonstrated trajectory, covering both the approach and interaction phases. When the task-relevant part changes its relative position across the target object instances (as illustrated by Target 1 and Target 2 in Fig. 5(b)) or the object poses vary within the workspace, such trajectory transfer cannot reliably preserve the spatial relationship required for successful task execution. As a

Method	Sweeping						Hanging						Pouring					
	Tool 1		Tool 2		Tool 3		Tool 1		Tool 2		Tool 3		Tool 1		Tool 2		Tool 3	
	Tgt 1*	Tgt 2	Tgt 1	Tgt 2	Tgt 1	Tgt 2	Tgt 1*	Tgt 2	Tgt 1	Tgt 2	Tgt 1	Tgt 2	Tgt 1	Tgt 2	Tgt 1*	Tgt 2	Tgt 1	Tgt 2
DINO-based Pose-Estimation Transfer	7/10	4/10	5/10	3/10	6/10	3/10	4/10	0/10	5/10	0/10	4/10	0/10	5/10	0/10	6/10	0/10	5/10	1/10
DITTO	8/10	4/10	6/10	3/10	7/10	3/10	5/10	0/10	5/10	0/10	4/10	0/10	5/10	1/10	6/10	0/10	5/10	2/10
TReF-6	6/10	5/10	5/10	4/10	4/10	4/10	4/10	3/10	5/10	4/10	5/10	2/10	6/10	6/10	7/10	4/10	6/10	5/10
OSPFI	9/10	9/10	9/10	8/10	10/10	8/10	9/10	8/10	9/10	9/10	8/10	8/10	9/10	7/10	9/10	8/10	8/10	7/10

TABLE I: Number of successful trials for different methods across object combinations. Tgt denotes Target. * Indicates the object combination used in the demonstration.

result, these two methods achieve low overall success rates, and their performance degrades to almost complete failure when the target object instance is changed. TReF-6 relies on fine-grained local regions to localize the task-relevant frame. However, it performs only a single-stage segmentation of these regions using Grounded SAM with fine-grained text prompts, which often results in imprecise segmentation and consequently unstable localization of the task-relevant frame. Table II reports the success rates of different methods on unseen object combinations. OSPFI achieves a success rate of 83.3%, substantially outperforming the baselines. These results demonstrate the generalization ability of OSPFI in part-level interaction tasks.

Method	Success Rate
DINO-based Pose-Estimation Transfer	27.3%
DITTO	30.0%
TReF-6	45.3%
OSPFI	83.3%

TABLE II: Overall success rates on unseen object combinations.

B. Comparison with Zero-Shot VLM-Based Manipulation

Besides the one-shot learning strategies, we also compare OSPFI with ReKep [32], a zero-shot VLM-based manipulation strategy. ReKep uses the VLM to identify semantic keypoints and generate relational constraints from visual observations and task instructions. These constraints are then optimized to generate multi-stage robot motions for task execution.

Under the same experimental setup, we compare ReKep and OSPFI on object combinations different from those used in the OSPFI demonstration, with 40 trials per task for each method. The results are shown in Fig. 6. OSPFI achieves higher success rates across all three tasks. While OSPFI uses point cloud alignment to ensure the reliability of the spatial relationship between objects, ReKep only relies on VLM reasoning and keypoint extraction to generate spatial relationship constraints. Due to the limited ability of the VLM to reason about spatial relationships and uncertainty in the overall constraint generation process, the generated sub-goal constraints may not accurately capture the required tool-target relationship, often resulting in inappropriate tool positions or orientations, which lead to task failure.

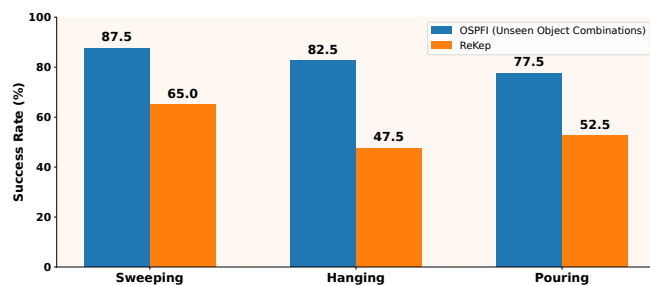


Fig. 6. Comparison of success rates between OSPFI and ReKep across three task scenarios.

C. Ablation Study

To verify the necessity of fine alignment, we remove this step and evaluate the method on both the object combination

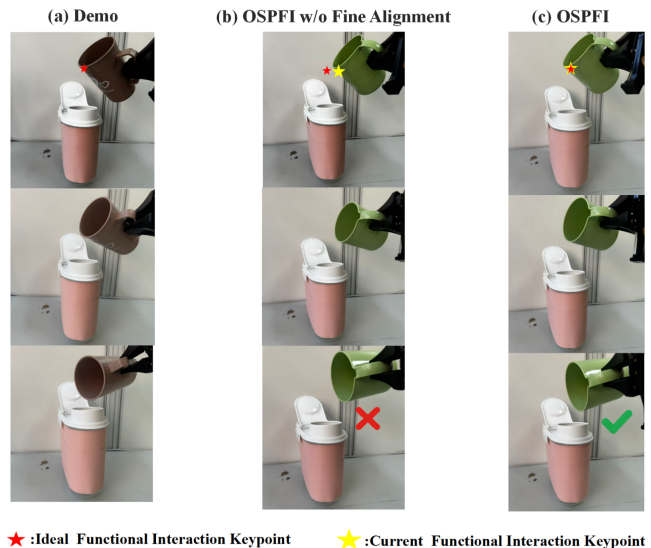


Fig. 7. An illustration of the effect of fine alignment taking the pouring task as an example. (a) The pouring process in the demonstration. (b) Without fine alignment, coarse alignment only aligns the tool part to an approximate position, leading to failure in the subsequent interaction. (c) The result after applying fine alignment, which ensures the critical interaction region and orientation of the tool are correctly aligned with the target part.

used in the demonstration and unseen object combinations, with 40 trials conducted for each setting.

The results are shown in Table III. Without fine alignment, the success rate decreases only slightly for the object combination used in the demonstration, but drops considerably on unseen object combinations, especially for the Hanging and Pouring tasks. This indicates that coarse alignment based on point cloud transformation can only provide a roughly reasonable initial tool pose by aligning the overall shape of the point clouds, but cannot reliably align the critical interaction region and tool orientation across tool instances with different shapes, leading to failure in subsequent task execution, as illustrated in Fig. 7. These results demonstrate the importance of fine alignment for generalization across object instances with different geometries.

Method	Sweeping		Hanging		Pouring	
	Demo.	Unseen	Demo.	Unseen	Demo.	Unseen
OSPFI	87.5%	87.5%	85.0%	82.5%	82.5%	77.5%
OSPFI w/o Fine Alignment	87.5%	77.5%	80.0%	57.5%	80.0%	55.0%

TABLE III: Success rates of OSPFI with and without Fine Alignment. Demo. denotes the object combination used in the demonstration, while Unseen denotes unseen object combinations.

VI. CONCLUSIONS

This paper presents OSPFI, a one-shot skill learning framework for part-level functional interaction. OSPFI represents interaction skills through part-level spatial relationships and transfers them to novel object instances using coarse-to-fine part alignment. Real-world experiments on multiple manipulation tasks demonstrate the effectiveness and generalization ability of the proposed method.

Limitations. Several limitations remain in OSPFI. First, fine alignment may fail when the tool part center, target part center, and functional interaction keypoint are collinear, since

a valid interaction plane cannot be constructed (though this case is rare). Moreover, fine alignment may also fail when the functional interaction keypoint cannot be correctly matched by RANSAC due to occlusion or other perception errors. In addition, the current pipeline is open-loop. Future work could incorporate online visual feedback for more robust closed-loop execution.

ACKNOWLEDGMENT

The authors used ChatGPT (OpenAI) to assist in generating and editing several small graphical elements used in Figs. 1, 2, and 4. All figure layouts and final illustrations were created and reviewed by the authors.

REFERENCES

- [1] S. Sontakke et al., “RoboCLIP: One demonstration is enough to learn robot policies,” in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023.
- [2] V. Patil et al., “Align-RUDDER: Learning from few demonstrations by reward redistribution,” in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 17531–17572.
- [3] M. Qin, J. Brawer, and B. Scassellati, “Rapidly learning generalizable and robot-agnostic tool-use skills for a wide range of tasks,” *Front. Robot. AI*, vol. 8, Art. no. 726463, 2021.
- [4] M. Y. Aoyama, S. Vijayakumar, and T. Narita, “Few-shot transfer of tool-use skills using human demonstrations with proximity and tactile sensing,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 9, pp. 9024–9031, 2025.
- [5] L. Schwenkel, M. Guo, and M. Bürger, “Optimizing sequences of probabilistic manipulation skills learned from demonstration,” in *Proc. Conf. Robot Learn. (CoRL)*, 2020, pp. 273–282.
- [6] O. Biza et al., “One-shot imitation learning via interaction warping,” in *Proc. 7th Conf. Robot Learn. (CoRL)*, 2023, pp. 2519–2536.
- [7] K. Zorina, J. Carpentier, J. Sivic, and V. Petrik, “Learning to manipulate tools by aligning simulation to video demonstration,” *IEEE Robot. Autom. Lett.*, vol. 7, no. 1, pp. 438–445, 2022.
- [8] N. Di Palo and E. Johns, “DINOBot: Robot manipulation via retrieval and alignment with vision foundation models,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 2798–2805.
- [9] M. Sieb, Z. Xian, A. Huang, O. Kroemer, and K. Fragkiadaki, “Graph-structured visual imitation,” in *Proc. Conf. Robot Learn. (CoRL)*, 2020, pp. 979–989.
- [10] N. Heppert, M. Argus, T. Welschhold, T. Brox, and A. Valada, “DITTO: Demonstration imitation by trajectory transformation,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024.
- [11] P. Vitiello, K. Drezekowski, and E. Johns, “One-shot imitation learning: A pose estimation perspective,” in *Proc. 7th Conf. Robot Learn. (CoRL)*, 2023, pp. 943–970.
- [12] C. Pan, B. Okorn, H. Zhang, B. Eisner, and D. Held, “TAX-Pose: Task-specific cross-pose estimation for robot manipulation,” in *Proc. 6th Conf. Robot Learn. (CoRL)*, 2023, pp. 1783–1792.
- [13] W. Liu, J. Mao, J. Hsu, T. Hermans, A. Garg, and J. Wu, “Composable part-based manipulation,” in *Proc. 7th Conf. Robot Learn. (CoRL)*, 2023, pp. 1300–1315.
- [14] A. Simeonov et al., “SE(3)-equivariant relational rearrangement with neural descriptor fields,” in *Proc. 6th Conf. Robot Learn. (CoRL)*, 2023, pp. 835–846.
- [15] C. Tang et al., “MimicFunc: Imitating tool manipulation from a single human video via functional correspondence,” in *Proc. 9th Conf. Robot Learn. (CoRL)*, 2025, pp. 4473–4492.
- [16] P. Gupta, H. Admoni, and A. Bajcsy, “Adapting by analogy: OOD generalization of visuomotor policies via functional correspondence,” in *Proc. 9th Conf. Robot Learn. (CoRL)*, 2025, pp. 5435–5449.
- [17] X.-H. Chen et al., “Deep demonstration tracing: Learning generalizable imitator policy for runtime imitation from a single demonstration,” in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, 2024, pp. 7586–7620.
- [18] C. Wang, C. Su, B. Sun, G. Chen, and L. Xie, “Extended residual learning with one-shot imitation learning for robotic assembly in semi-structured environment,” *Front. Neurobot.*, vol. 18, 2024.
- [19] X. Zhang and A. Boularias, “One-shot imitation learning with invariance matching for robotic manipulation,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.
- [20] Y. Liu, J. Mao, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, “One-shot manipulation strategy learning by making contact analogies,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 15387–15393.
- [21] Y. Wang and E. Johns, “One-shot dual-arm imitation learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.
- [22] Z. Wang and G. Tian, “Task-oriented robot cognitive manipulation planning using affordance segmentation and logic reasoning,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 12172–12185, 2024.
- [23] F.-J. Chu, R. Xu, and P. A. Vela, “Learning affordance segmentation for real-world robotic manipulation via synthetic images,” *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 1140–1147, 2019.
- [24] D. Hu, T. Sun, P. Xie, S. Chen, H. Yang, and G. Wang, “Variation-robust few-shot 3D affordance segmentation for robotic manipulation,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 2, pp. 1672–1679, 2025.
- [25] H. Geng, S. Wei, C. Deng, B. Shen, H. Wang, and L. J. Guibas, “SAGE: Bridging semantic and actionable parts for generalizable articulated-object manipulation under language instructions,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.
- [26] Y. Yin et al., “PartInstruct: Part-level instruction following for fine-grained robot manipulation,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [27] P. Florence, L. Manuelli, and R. Tedrake, “Self-supervised correspondence in visuomotor policy learning,” *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 492–499, 2020.
- [28] L. Manuelli, Y. Li, P. Florence, and R. Tedrake, “Keypoints into the future: Self-supervised correspondence in model-based reinforcement learning,” in *Proc. Conf. Robot Learn. (CoRL)*, 2020, pp. 693–710.
- [29] Y. Liu et al., “SFK: Shape- and function-grounded keypoint representation for sequential manipulation,” *CAAI Trans. Intell. Technol.*, 2026.
- [30] L. Manuelli, W. Gao, P. Florence, and R. Tedrake, “kPAM: KeyPoint affordances for category-level robotic manipulation,” in *Robotics Research: 19th Int. Symp. ISRR*, pp. 132–157, 2022.
- [31] Z. Qin, K. Fang, Y. Zhu, L. Fei-Fei, and S. Savarese, “KETO: Learning keypoint representations for tool manipulation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2020, pp. 7278–7285.
- [32] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” in *Proc. 8th Conf. Robot Learn. (CoRL)*, 2024, pp. 4573–4602.
- [33] Q. Liu, E. Chen, L. Gao, C. Liang, and H. Liu, “Energy-guided temporal segmentation network for multimodal human action recognition,” *Sensors*, vol. 20, no. 17, Art. no. 4673, 2020.
- [34] P. Sun et al., “Going denser with open-vocabulary part segmentation,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 15453–15465.
- [35] J. Choi, S. Lee, S. Lee, M. Lee, and H. Shim, “Understanding multi-granularity for open-vocabulary part segmentation,” in *Adv. Neural Inf. Process. Syst.*, vol. 37, 2024.
- [36] S. Liu et al., “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024.
- [37] A. Kirillov et al., “Segment Anything,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 4015–4026.
- [38] Y. Eldar, M. Lindenbaum, M. Porat, and Y. Y. Zeevi, “The farthest point strategy for progressive image sampling,” *IEEE Trans. Image Process.*, vol. 6, no. 9, pp. 1305–1315, 1997.
- [39] M. Oquab et al., “DINOv2: Learning robust visual features without supervision,” *arXiv preprint arXiv:2304.07193*, 2023.
- [40] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 13, no. 4, pp. 376–380, 1991.
- [41] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [42] H. Fang et al., “AnyGrasp: Robust and efficient grasp perception in spatial and temporal domains,” *IEEE Trans. Robot.*, vol. 39, no. 5, pp. 3929–3945, 2023.
- [43] Y. Ding, S. Wang, and T. Fitzgerald, “TReF-6: Inferring task-relevant frames from a single demonstration for one-shot skill generalization,” in *Proc. 9th Conf. Robot Learn. (CoRL)*, 2025, pp. 5129–5150.